

寫作導向式參考資料整理

讀了論文就不要浪費！整理到所需要章節！

Refactoring

Abstract

- Refactoring approaches is based on the idea to refine the code by appropriate refactoring methods
- according to the Martin book refactoring has some advantages.

Refactoring approaches is based on the idea to refine the code by selecting appropriate refactoring methods and then to assemble them with a intuition. According to the Martin book [Fowler1999b], refactoring is important due to "improves the design of software", "makes software easier to understand", "help you find bugs", "helps you program faster". There are two definitions contribute to the refactoring in the Martin book.

- because refactoring progress always comprise of the different refactoring methods
- all methods show you how to do separately, and don't tell you anything about the notion of abstract.
- we propose a generic component model that according to the polymorphic conception
- Polymorphism to the ability of a component to exhibit different behaviors under different contexts.

[from JYC] Therefore, we divide the context of a component conceptually into two parts: *hard-wired* and *polymorphic*. The previous corresponds to those components

used directly by that component, and the relation is usually not replaceable without combing through the component internals.

- propose some metrics to measure reusability in our thesis
- reusability can be used to apply the degree of measurement features
- we expect to the metric that can be effectively applied in the component

we will propose some metrics to measure reusability in our thesis. Reusability can be used to apply the degree of measurement features that they are reused in building applications. We expect to the metrics that can be effectively applied in the component.

- we survey current refactoring methods in common using
 - we also propose some metrics for our component-based refactoring compare with common refactoring methods
-

1. Introduction

- Modern software systems become more and more large-scale.
- Making design is a very important process to frequent requirement changes.
- The OO software development approach is very different from the traditional approach.

Modern software systems become more and more large-scale, complex and uneasily controlled, resulting in high development cost, low productivity, unmanageable software quality and high risk to modify the code. Making design is a very important process to frequent requirement changes. One of the most promising solutions today is the Object-Oriented language. This language is based on the idea that includes inheritance, polymorphism. The Object-Oriented software development approach is very different from the traditional approach in which software systems can only be implemented from scratch. These Object-Oriented component always can be developed by different developers using different language and different platforms.

[Cai2000a] These Object-Oriented component always can be developed by different developers using different language and different platforms;

- Reverse engineering is an ongoing process in the development of any successful software system
- system developers have benefits to keep track of the design
- a reverse engineering technique should explore whatever information is available
- in OO development, changes are accomplished by means of so-called refactorings
- To reverse engineer OO software, we must identify which refactorings have been applied and help us understanding how and the system has evolved.

[Serge1999a] Reverse engineering is an ongoing process in the development of any successful software system as it evolves in response to changing requirements. Therefore, system developers may benefit a lot from reverse engineering techniques that keep track of the design as it drifts from that of earlier versions.

problems of current refactoring practices [main]

- software in a real-world environment is need to evolve; As the software is enhanced, modified and adapted to new requirement, the code becomes more and more complex.
- because of this, the major part of the total software development cost is devoted to software maintenance
- Better software development methods and tools don't solve this problems
- There many methods to solve this problems; formalisms: program slice, graph transformations, software metrics, formal concept analysis
- *use component-based concept to develop system environment.*
- *according to the difference of resource and present the difference graph.*
- *to choose right positions you want to refactorings.*
- *introduce refactoring into component-based environment reusable and easier to understand.*
- *Due to refactoring, we will add behaviors in my program.*

2. Background Review

We will discuss Common Component-based Conception in this paragraph

- Why component rather than OO
- Interface and black-box is not sufficient, just like encapsulation and inheritance for OO – necessary but primitive.
- We proposed polymorphism as the even more crucial aspects for components, added to the core/basic design vocabulary.
- Reverse engineering tools
 - Classification of component relations

Refactoring Method

Refactoring is complex. Much of the complexity is due to the interrelationships among classes. For example, when a class is moved and a subclass/superclass relationship is changed to a container/component relationship, checks and structural changes are often needed to retain inherited behavior and satisfy type requirements on assignments and function calls. As a system grows, these task by hand becomes more complicated. The high costs of developing software motivate the reuse and evolution of existing software. Approaches that support reuse address one or more of the following four important aspects:

1. finding a reusable component.
2. understanding the component.
3. modifying a component or a set of components.
4. composing the components together.

Software restructuring relates to each of the four important aspects of reuse listed above.

Most of our refactorings are low level, such as creating or deleting a class, variable, or function; changing attributes of variables and functions, such as their access permissions (e.g., public or protected) and function arguments; or moving variables and functions between classes. A smaller set of high-level refactorings are used for operations such as creating an abstract superclass, simplifying a class by means of subclassing and simplifying conditionals, or splitting off part of an existing class to create a new, reusable component class (often converting between inheritance and delegation or aggregation). The more complex refactorings are defined in terms of the low-level refactorings. Our approach was motivated by concern for automated support and safety, which I discuss later.

According to the Metrics to express the programming efficiency.

our component model advantages

- refactoring is not only for OO but others
 - system partition; architecture design
-

3. Our Motivation and Objective

- Fowler book points to the different refactoring methods used by tools. Nevertheless, each methods handle different situations.
- There are not a generic conception to represent the refactorings

According to the Fowler book, there are a lot of refactoring methods used by tools. Nevertheless, every methods handle different situations. For this reason, we propose a generic component framework to adopt different generic refactoring methods, and restructure a program can make it easier to understand the design of a program and can assist in finding reusable components.

Changing an object-oriented system requires changing the abstractions embodied in existing components and the relationships among those components. This involves structural changes such as moving components and relationship between components and partitioning a complicated components into several components. When a structural change is made to a component or set of components, corresponding changes may also be needed elsewhere in a program.

Objective to the semi-automate support for refactoring

- so we propose a generic component framework to adopt different generic refactoring methods, and restructure a program can make it easier to understand the design of a program.

One approach to achieving meaningful reductions in software costs is to acquire an existing software system rather than developing a new one. Restructuring a program can make it easier to understand the design of a program and can assist in finding reusable components.

An approach for providing automated support for the restructuring of component-based object diagram (high-level) and low level object programs. When we restructure component-based object, there is a scale that show the restructure level by refactoring. Finally, People can know what is the right restructuring method

depend on the metrics. Introduction the java system codes into our Component-based Refactoring System and show where wants to improve.

4. Proposed Method

- what is the important step in decomposing the system into component.
- what kinds of functions on these protocols support
- in practice of component-based , what kinds of lacks in component-based development
- based on the lacks, we propose a component architecture to improve it [main]
- more detail on the architecture
- say something to prove our architecture is enough.

show our generic component model

- ...

A Generic Component Model

A component is any identifiable software artifact that provides knowledge or services through its interface. The different types of components, and may include high-level architectures, concrete implementations such as Java packages, classes and methods. Component can be combined according to rules associated with their interfaces to form larger components. Consequently, the forms of interfaces and the ways components are combined depend on the component types. ;Note that when we use the term interface, we also refer to the semantic implications behind it, that is, the contract.

Context, Polymorphism, and domain

Holistically, a system consists of multiple components interconnected with each other. Atomistically, a component functions by interacting with its **context**, where the context can be any medium through which the component can obtain services it requires. In this thesis we refer **polymorphism** to the ability of a component to *exhibit different behaviors under different contexts*. Therefore, we divide the context of a component conceptually into two parts: *hard-wired* and *polymorphic*. The former corresponds to those components *used directly* by that component, and the relation is

usually not replaceable without combing through the component internals. The latter, on the other hand, corresponds to those components that are replaceable by design, and there usually exist some language mechanisms to indicate the intension.

Accordingly, the interface of a component is divided conceptually into its **domain**, which refers to the primary functionality of the component, and **polymorphism interface** (or polymorphism for short when no ambiguity arises), which indicates the part of interface used for behavior tailoring. We say a component is polymorphic if it provides some polymorphism interface. Because domain characterizes what a component does, we often use domain and component interchangeably.

explain the context and polymorphism in our component model

- ...

Component Model

General Component Model

At the most basic level, components are parts that can be composed with others to build something bigger. **component-based development(CBD)** an aproach to software development in which all artifacts - from executable code to interface specifications, architectures, and business models; and scaling from complete applications and systems down to small parts - can be built by assembling, adapting, and "wiring" together existing components into a variety of configurations.

Why use Refactoring methods in the programming.

-

Component-based Metrics

5. Experiments

6. Conclusion and Future Trends

- development an abstract component model to express software artifacts

- basic concepts introduced in the model; context, polymorphism, collaboration etc..
- we also presented the maximal reusability principle.

We have developed an abstract component model that is capable of representing software artifacts of various types at different abstraction levels in a uniform way. We believe that the basic concepts introduced in the model - context, domain, polymorphism, collaboration, and management - provide a more versatile yet concise vocabulary in describing software designs. In terms of the model, we have also presented the maximal reusability principle as a tradeoff between component applicability and usability.

- attempt to provide an immediately useful tool not only to help software developers in performing design "refactoring", but also to help them

The framework is our attempt to provide an immediately useful tool not only to help software developers in performing design “refactoring” quickly and more objectively, but also to help them in sieving through literatures more efficiently and applying them more soundly. In other words, the framework complements and reconciles existing approaches and techniques.

- Our near-term goals is to have a semi-automatic graphic refactoring system

The near-term goal is to have a **semi-automatic graphic refactoring system** that can assist programmers to manage Java programs according to the component model. The framework represents the core of our ongoing research toward a streamlined CASE tool that can manage designs and implementation in a systematic way. With annotations, one can browse high-level architectures through low-level code fragments in a uniform way, thus promoting traceability. Effective refactoring browser for designs can be developed accordingly.

Acknowledgement

References

WebSite

- [1] <http://www-106.ibm.com/developerworks/opensource/library/os-ecref/>
- [2] <http://www.refactoring.com/tools.html>

Paper

- [Fowler2002a] Martin Fowler "Tutorials: Refactoring," 2002
- [Fowler1999b] Martin Fowler, Refactoring: Improving the Design of Existing Programs, Addison-Wesley, 1999, To Appear.
- [Mens2003a] T. Mens and A. V. Deursen. "Refactoring: Emerging Trends and Open Problems," Oct 2003
- [Mens2003b] T. Mens, S. Demeyer, B. D. Bois, H. Steven, P. V. Gorp "Refactoring: Current Research and Future Trends," 2003
- [Serge1999a] Serge Demeyer, Stephane Ducasse, Oscar Nierstrasz "Finding Refactoring via Change Metrics," 1999
- [Mens2003c] Tom Mens , Serge Demeyer , and Dirk Janssens "Formalising Behaviour Preserving Program Transformations", 2003
- [Opdyke1992a] Opdyke, W. "Refactoring Object-Oriented Frameworks," 1992
- [Roberts1997a] Don Roberts, John Brant, and Ralph Johnson "A Refactoring Tool for Smalltalk," 1997
- [Suny2004a] Gerson Suny e, Damien Pollet, Yves Le Traon, and Jean-Marc J ez equel "Refactoring UML Models," 2004
- [Scott2003a] Scott Grant, James R. Cordy "An Interactive Interface for Refactoring Using Source Transformation," 2003
- [Rose1997a] Linda H. Rosenberg, Lawrence E. Hyatt "Software Quality Metrics for Object-Oriented Environments," 1997
- [Eun2001a] Eun Sook Cho, Min Sun Kim, Soo Dong Kim "Component Metrics to Measure Component Quality," 2001
- [Cai2000a] Xia Cai, Michael R. Lyu, Kam-Fai Wong, and Roy Ko "Component-Based Software Engineering: technologies, Development Frameworks, and Quality Assurance Schemes," 2000
- [Szy1998a] C. Szyperski, Component Software: Beyond Object-Oriented Programming, Addison-Wesley, 1998
- [Tar1999a] P. Tarr and S.M. Sutton. N Degrees of Separation: Multi-Dimensional Separation of Concerns. ICSE'99. 107-119.